BidSense: Project Proposal

AI-Powered Bid and Proposal Response Engine CUST Hackathon 2026 · Problem Statement #1 (TEKROWE)

Project	BidSense
Problem	#1, AI Bid and Proposal Response Engine
Demo date	13 June 2026
Repository	github.com/Qubit1010/bid-engine

Team

Member	Role
Aleem Ul Hassan	Data and ML Integration
Anas Khan	Backend Developer
Umer Khatab	Frontend

1. Executive summary

Bid and proposal teams spend 60 to 80 percent of their time reading tenders and assembling responses, and a single missed mandatory clause can disqualify an entire bid. BidSense reads a tender the way a senior bid manager does: it extracts every requirement, checks each one against the company's capability library with cited evidence, scores the bid's win probability with an explainable machine-learning model, and drafts a compliant proposal where every claim is traceable to a real record. The output is a clear GO, CONDITIONAL GO, or NO-GO decision plus a review-ready draft, produced in minutes instead of days.

The prototype is end to end and working on the hackathon's own dataset. It runs offline from a warmed response cache, so it survives venue Wi-Fi and API outages, and it ships with a live in-app validation page so judges see evidence rather than claims.

2. Problem statement

Responding to a public tender is slow, manual, and risky:

- A typical 15 to 80 page RFP takes two to four working days to reach a first draft.
- Mandatory requirements are scattered across eligibility clauses, submission rules, and annexes, and humans miss them.
- Companies commit teams to tenders they cannot win, and skip tenders they could have won, because nobody can quantify win probability before the effort is spent.

The cost is not only the writing time. It is disqualification on a technicality and wrong pursue/pass decisions.

3. Proposed solution

Upload a tender (PDF or DOCX). BidSense runs a four-stage pipeline and returns a complete, auditable bid workspace:

1. **Extract** every requirement, evaluation criterion, deadline, and budget, with source pages. 2. **Match** each requirement against the company's capability library, returning PASS, PARTIAL, or GAP with cited evidence. 3. **Score** win probability with a trained model, and issue a GO / CONDITIONAL GO / NO-GO decision with a written rationale. 4. **Draft** the full proposal, with every factual claim cited to a real capability record, then export to Word.

We demonstrate the engine on three tenders that produce three honest answers: a solar project (GO), a hospital IT project (CONDITIONAL GO, blocked by one closable certification gap), and a road project (NO-GO, four genuine eligibility gaps). A tool that flatters every bid is useless. Ours says no when it should.

4. How it works

```
Upload (PDF / DOCX)
   -> Ingest        text + page map
   -> Extract       LLM structured extraction + regex validators (dates, PKR, %)
   -> Match (RAG)   retrieve capability evidence -> PASS / PARTIAL / GAP
   -> Win model     score estimator (Stage B) -> trained classifier -> GO / NO-GO
   -> Draft         section-by-section, every claim cited [CAP-xxx] / [CO-PROFILE]
   -> Export        proposal DOCX + compliance matrix CSV
```

The matcher classifies each requirement into one of four types (procedural commitments, verifiable bidder attributes, experience thresholds, and forward delivery obligations) and judges each type by the right standard. This is what lets it pass a credible delivery commitment while correctly failing a certification the company does not hold.

5. Key features (mapped to the deliverables)

Problem-statement deliverable	BidSense feature
Ingest RFP/RFQ/Tender documents	PDF and DOCX ingestion with page mapping
Extract requirements and criteria	LLM extraction validated against a typed schema
Match against a capability library	Hybrid RAG retrieval with cited evidence per requirement
Identify compliance gaps	PASS / PARTIAL / GAP matrix, mandatory gaps flagged
Score win probability	Trained XGBoost model with SHAP explanations
GO / NO-GO recommendation	Decision logic plus an auto-generated decision memo
Draft the proposal response	Citation-grounded section drafting with approve / edit / regenerate
Demonstrate effort reduction	Pipeline time versus a two-day-plus manual baseline, shown per workspace
Per-tender workspaces	One workspace per uploaded document, fully persisted

6. Innovation and differentiation

Most AI proposal tools are a text box over a chatbot. BidSense is an auditable decision system.

- **Anti-hallucination by contract.** The draft can only cite capability IDs that exist. Fabricated citations are rejected and unit-tested.
- **Explainable win probability, not a guess.** The probability comes from a model trained on the 120-bid history with SHAP per-feature explanations. The language model never produces the number.
- **Intellectual honesty built in.** We publish an ablation study inside the app showing exactly what the model learned, instead of hiding the dataset's limitations.
- **It says no.** The system declined a road tender with four genuine eligibility gaps.
- **Offline-capable.** A response cache lets the full demo run with no network and no API keys.

7. Technical architecture and stack

```
Next.js 16 + TypeScript + Tailwind 4    (frontend, port 3000)
        |  server-side /api proxy (no CORS)
FastAPI + Python 3.11                    (backend, port 8000)
        |  ingest · extract · rag/match · winprob · draft · export
SQLite                                   (workspaces, requirements, matches, drafts)
```

- **LLM layer:** OpenAI as the primary provider with a Claude fallback, wrapped in a disk cache keyed on the prompt, which both controls cost and enables full offline replay.
- **Retrieval:** embeddings over the 50-record capability library with hybrid scoring (no external vector database needed at this scale).
- **Win model:** XGBoost with L2-regularized logistic regression as a benchmark, selected by stratified 5-fold cross-validated ROC-AUC, with SHAP explanations.
- **Export:** python-docx proposal generation and a compliance matrix CSV.

8. Data and methodology

We worked from the three provided datasets: a 120-row bid history, a 50-row capability library, and an evaluation-criteria taxonomy. The taxonomy sheet referenced in the problem statement was missing from the sample file, so the team synthesized a 16-entry taxonomy to fill the gap, which we disclose openly.

Exploratory analysis (reproducible in `eda/generate_eda.py`, with a full `EDA_REPORT.md` and figures surfaced on the in-app Validation page) found that the bid outcome is driven almost entirely by the awarded evaluation score, with a clean threshold near 70 (correlation with winning of +0.86, versus under 0.10 for every other feature). Because the awarded score is only known after evaluation, it cannot be a live input. BidSense therefore estimates the expected score before submission (Stage B) from compliance coverage, sector win-rate priors, and budget alignment, then maps that estimate to win probability with the trained model.

9. Validation and reliability

- 32 automated tests passing, surfaced live on the in-app Validation page.
- Model metrics reported out of fold (cross-validated) to avoid optimistic bias, with a confusion matrix and the score-to-win curve.
- A published ablation study: remove the score feature and accuracy collapses to roughly a coin flip, which proves the pre-bid features alone carry no signal and explains why the cross-validated accuracy is high without being memorization.
- Human in the loop by design: analysts can override any match status and approve every drafted section before export.

10. Feasibility and scalability

The prototype runs on a laptop today. The cost per processed bid is a few cents of language-model and embedding spend, which keeps gross margin software-grade. Public procurement is typically 15 to 20 percent of GDP, and proposal-response software is already an established global category (Responsive, Loopio), with no localized player handling PPRA formats, PKR amounts, and local eligibility rules such as PEC categories and ISO requirements. The natural beachhead is Pakistani IT-services and construction firms that bid on government and donor-funded tenders every month.

A realistic path to product:

1. **Now:** single-company workspace, three demo verticals, full pipeline working. 2. **Next quarter:** a capability-library builder that ingests past proposals automatically, a PPRA tender feed, and a win/loss feedback loop that retrains the model on each closed bid. 3. **Year one:** multi-tenant SaaS with team workflows and pricing intelligence from historical award data.

11. Team and responsibilities

Member	Role	Owns
Aleem Ul Hassan	Data and ML Integration	Datasets and EDA, win-probability model and score estimator, RAG matching, pipeline orchestration
Anas Khan	Backend Developer	FastAPI services, ingestion and extraction, export, persistence, test suite
Umer Khatab	Frontend	Next.js workspaces, compliance and draft UI, win-probability dashboards, validation page

12. Summary

BidSense turns a tender into a cited proposal draft and an explainable GO/NO-GO decision in minutes. It is technically complete, validated openly, resilient offline, and built on the hackathon's own data. The team built the full system end to end, and the same engine generalizes to any company that answers tenders.